

(续)

操作 (按时间排序)	持有线程	等待线程	value	wakeup	剩余资源	注 释
新来 2 个线程请求资源	7	3	0	3	3	直接消耗 value, 无须等待
1 个等待线程获取资源	8	2	0	2	2	等待线程消耗 wakeup
新来 1 个线程请求资源	8	3	-1	2	2	虽然 wakeup 不为 0, 但已经预留给之前等待的线程。value 再次被减到负数, 新的线程必须等待
1 个线程释放资源	7	3	0	3	3	
1 个线程释放资源	6	3	1	3	4	此时等待的线程都已经预留对应资源, 释放的资源直接加到 value
6 个线程释放资源	0	3	7	3	7	现在没有线程拿着资源
3 个等待线程获取资源	3	0	7	0	10	等待线程消耗 wakeup
3 个线程释放资源	0	0	10	0	10	所有资源被释放

下面将详细分析信号量的两个操作的实现。首先分析信号量的 wait 操作。线程获取该信号量的互斥锁后 (第 10 行), 先将 value 的值减 1 (第 11 行)。若其值大于等于 0, 则代表有空闲的资源, wait 操作成功。如果 value 的值小于 0, 那么它不再表示当前可用的资源数量, 我们需要检查 wakeup 来判断是否还有空闲的资源可以消耗。当 wakeup 大于 0 时, 代表还有空闲的资源可以消耗。而如果 wakeup 也为 0, 就需要用 10.2 节实现的 cond_wait 将当前线程挂起等待 (第 14 行)。

小思考

在 wait 操作中, 为何使用 do...while 而非 while? 有什么差别吗?

这里使用 do...while 而非 while, 是为了提供有限等待的保证, 确保调用 signal 后立刻调用 wait 的线程不会直接拿走资源, 而是交给正在等待的线程。下面通过一个例子进行详细说明。

假设线程 0 与线程 1 两个线程使用信号量进行同步, 它们的执行顺序如表 10.3 所示。在 T_0 时刻, 信号量的 value 为 0。此时线程 0 调用 wait 操作挂起等待。随后在 T_1 时刻, 线程 1 调用 signal 操作给 wakeup 加 1, 表明此时有可用资源, 并尝试唤醒线程 0。由于调用 wakeup 的时间与线程 0 被调度执行有一定的时间差, 线程 0 在 T_3 时刻才真正被唤醒。在 T_2 时刻, 线程 1 再次调用 wait。在使用 while 的情况下, 线程 1 将可能在线程 0 被唤醒前 (T_3) 抢先一步拿到该资源。如此往复, 线程 1 可能会一直自己生产、消耗信号量, 而线程 0 一直无法执行。因此, 实现中使用 do...while, 要求线程在检查 wakeup 之前调用 cond_wait 并排队, 避免造成线程 0 一直无法被唤醒。

表 10.3 使用信号量的某一执行顺序

时刻	线程 0	线程 1
T_0	wait(&S) 挂起等待	

(续)

时刻	线程 0	线程 1
T_1		signal(&S)
T_2		wait(&S) 拿到资源
T_3	被唤醒，发现没有可用资源	

对应地，在执行 signal 操作时，线程首先将 value 的值加 1（第 24 行），然后判断是否还有线程等待在该信号量上（第 25 行）。如果有则增加 wakeup（第 26 行），并使用 cond_signal 唤醒其中的一个线程获取资源（第 27 行）。

读者可以思考一下，代码片段 10.10 的实现能够保证按照调用 wait 的顺序拿到资源吗？实际上，以上实现并不能保证线程按照调用 wait 的顺序依次拿到资源。考虑这样的情况，系统中存在线程 0、线程 1 与线程 2，它们的执行顺序如表 10.4 所示。在 T_0 时刻，信号量的 value 为 0。此时，线程 0 调用 wait 操作挂起等待。在 T_1 与 T_2 时刻，线程 1 与线程 2 分别调用 signal 操作增加可用资源。此时信号量的 value 为 1，而 wakeup 也为 1。在 T_3 时刻，线程 1 调用 wait 操作后，无须等待便可直接拿到资源。线程 0 则到稍晚的 T_4 时刻被唤醒之后才能拿到资源。上述例子中，线程 0 虽然比线程 1 更早调用 wait 操作，却更晚拿到资源。但这并不会破坏有限等待的保证，出现上述情况的前提是信号量此时能够满足所有挂起线程的使用，value 才有可能被加回正数。因此即使后来的线程可能抢先一步拿到资源，但是挂起的线程也能够在被唤醒后拿到资源。所以不会出现阻塞挂起线程的情况。

表 10.4 使用信号量的三个线程的执行顺序

时刻	线程 0	线程 1	线程 2
T_0	wait(&S) 挂起等待		
T_1		signal(&S)	
T_2			signal(&S)
T_3		wait(&S) 拿到资源	
T_4	被唤醒，拿到资源		

10.4 读写锁的实现

本节主要知识点

- ❑ 如何正确实现偏向读者的读写锁？
- ❑ 如何正确实现偏向写者的读写锁？

为了解决读者写者问题，提升读者之间的并行度从而提升效率，我们在第 9 章还介绍了另一种同步原语：读写锁。与互斥锁不同，读写锁仅仅保证读者与写者之间的互斥，而允许多个读者同时进入读临界区。读写锁允许多个读者同时进入读临界区带来了倾向性问题。假设在某一个时刻，已经有一些读者在临界区中。此时有一个写者与一个读者同时申请进入临界区，我们是否应该允许读者直接进入读临界区？如果允许读者直接进入，那么写者会被一直阻塞直到没有任何读者，甚至陷入无限等待。如果等之前的读者离开临界区后，先允许写者进入，再允许读者进入，这样虽能避免写者陷入无限等待，但同时也减少了读者的并行性。我们称前者为偏向读者的读写锁，称后者为偏向写者的读写锁。下面我们将分别介绍这两种读写锁的实现。

10.4.1 偏向读者的读写锁

代码片段 10.11 展示了一种偏向读者的读写锁。读写锁的结构体包含一个用于表示当前读者数目的计数器 `reader_cnt`，其在初始化时被设置为 0。这种实现的基本思路是通过该计数器记录当前在读临界区的读者数量。当该计数器为 0 时，代表没有读者在读临界区内。此时，写者可以进入写临界区。反之，写者无法进入写临界区，必须要等待所有读者退出读临界区。除此之外，该结构体还包含用于控制读者对 `reader_cnt` 进行更新的互斥锁 `reader_lock`，以及用于控制读者与写者、写者与写者之间互斥的互斥锁 `writer_lock`。

代码片段 10.11 偏向读者的读写锁

```
1 struct rwlock {
2     volatile int reader_cnt;
3     struct lock reader_lock;
4     struct lock writer_lock;
5 };
6
7 void lock_reader(struct rwlock *lock)
8 {
9     lock(&lock->reader_lock);
10    lock->reader_cnt++;
11    if (lock->reader_cnt == 1)           // 第一个读者
12        lock(&lock->writer_lock);
13    unlock(&lock->reader_lock);
14 }
15
16 void unlock_reader(struct rwlock *lock)
17 {
18    lock(&lock->reader_lock);
19    lock->reader_cnt--;
20    if (lock->reader_cnt == 0)           // 最后一个读者
21        unlock(&lock->writer_lock);
22    unlock(&lock->reader_lock);
```

```
23 }
24
25 void lock_writer(struct rwlock *lock)
26 {
27     lock(&lock->writer_lock);
28 }
29
30 void unlock_writer(struct rwlock *lock)
31 {
32     unlock(&lock->writer_lock);
33 }
```

读者在上锁时 (`lock_reader`)，需要使用 `reader_lock` 保证对于 `reader_cnt` 更新的原子性 (第 9 行)。如果当前读者是第一个读者 (第 11 行)，则还需要获取 `writer_lock` (第 12 行)，从而保证读者与写者的互斥并阻塞后续写者。注意，这里使用互斥锁而非原子操作保证对 `reader_cnt` 更新的原因是，第 9 行的上锁操作 (`reader_lock`) 还用于在有写者执行写临界区时阻塞后续读者。

同样，如果读者在放锁 (`unlock_reader`) 时发现自己是最最后一个读者 (第 20 行)，则还需要释放 `writer_lock` (第 21 行)，用于取消对写者的阻塞。后续读者在进入读临界区之前，必须再次获取 `writer_lock`。

对于写者来说，只需要在加锁和放锁时操作对应的 `writer_lock` 即可，这是由于当有读者在读临界区内时，`writer_lock` 一定被读者持有，因此写者只要操作 `writer_lock` 就可以保证与读者以及其他写者的互斥。

你可能会疑问，我们想优化读者，怎么读者的加锁和放锁流程相比于互斥锁更加复杂了？读者还需要单独的一个读者锁，这是不是与“读者不互斥”的目标冲突了？虽然读者加锁和放锁的流程在这个实现中确实较互斥锁更为复杂，但是读写锁的核心目标不是简化读者加锁和放锁的流程，而是增加读者之间的并行度。使用读写锁，不同的读者现在可以同时处于读临界区，而不是必须串行地一个个进入读临界区，这将大大提高读操作多的应用程序的性能。至于额外的读者锁，在临界区开始之前就已经执行了放锁操作，其保护的是对读写锁元数据的修改，而不是接下来的读临界区。

10.4.2 偏向写者的读写锁

偏向写者的读写锁实现起来较偏向读者的读写锁更为复杂。代码片段 10.12 展示了一种偏向写者的读写锁。由于偏向写者的读写锁需要在有新的写者到达时阻塞后续读者，因此除了用于记录当前读者数量的 `reader_cnt` 之外，还添加了一个布尔变量 `has_writer` 来表示当前是否有写者到达。当有写者期望获取锁时，即使其还未真正进入临界区，也会将 `has_writer` 设置为 `TRUE`。读者将通过判断 `has_writer` 的值来决定是否需要等待前序写者。由于写者也会操作读写锁中的元数据 (包括 `has_writer`

与 reader_cnt), 因此偏向写者的读写锁提供了一把读者和写者共享的锁 rwlock->lock, 要求读者与写者在操作元数据前获取该锁。

代码片段 10.12 偏向写者的读写锁

```
1 struct rwlock {
2     volatile int reader_cnt;
3     volatile bool has_writer;
4     struct lock lock;
5     struct cond reader_cond;
6     struct cond writer_cond;
7 };
8
9 void lock_reader(struct rwlock *rwlock)
10 {
11     lock(&rwlock->lock);
12     while(rwlock->has_writer == TRUE)
13         cond_wait(&rwlock->writer_cond, &rwlock->lock);
14     rwlock->reader_cnt++;
15     unlock(&rwlock->lock);
16 }
17
18 void unlock_reader(struct rwlock *rwlock)
19 {
20     lock(&rwlock->lock);
21     rwlock->reader_cnt--;
22     if (rwlock->reader_cnt == 0)
23         cond_signal(&rwlock->reader_cond);
24     unlock(&rwlock->lock);
25 }
26
27 void lock_writer(struct rwlock *rwlock)
28 {
29     lock(&rwlock->lock);
30     while(rwlock->has_writer == TRUE)
31         cond_wait(&rwlock->writer_cond, &rwlock->lock);
32     rwlock->has_writer = TRUE;
33     while(rwlock->reader_cnt > 0)
34         cond_wait(&rwlock->reader_cond, &rwlock->lock);
35     unlock(&rwlock->lock);
36 }
37
38 void unlock_writer(struct rwlock *rwlock)
39 {
40     lock(&rwlock->lock);
41     rwlock->has_writer = FALSE;
42     cond_broadcast(&rwlock->writer_cond);
43     unlock(&rwlock->lock);
44 }
```

读者在申请进入临界区 (`lock_reader`) 时, 如果观察到 `has_writer` 被置为 `TRUE` (第 12 行), 则不能进入临界区, 需要使用条件变量挂起等待 (第 13 行)。确保没有写者等待时, 读者在增加读者计数器 `reader_cnt` 并放锁后便可进入读临界区 (第 14、15 行)。

对应地, 在写者需要进入临界区 (`lock_writer`) 时, 也需要通过 `has_writer` 判断是否有其他写者在临界区内 (第 30 行)。如果存在其他写者, 则需要使用条件变量挂起等待 (第 31 行)。当没有其他写者时, 当前写者就可以将 `has_writer` 设置为 `TRUE` (第 32 行), 并使用条件变量等待之前所有的读者离开读临界区 (第 33、34 行)。最后, 释放保护元数据的锁即可进入写临界区 (第 35 行)。

读者在放锁时 (`unlock_reader`), 除了需要减少读者的计数器 `reader_cnt` 值 (第 21 行), 还需要判断自己是不是最后一个读者 (第 22 行)。如果是最后一个读者, 还需要使用 `cond_signal` 来唤醒等待在 `reader_cnt` 上的写者 (第 23 行)。

写者在放锁时 (`unlock_writer`), 除了设置 `has_writer` 为 `FALSE` (第 41 行), 还应当使用 `cond_broadcast` 将等待在 `has_writer` 上的读者和写者都唤醒 (第 42 行)。注意这里使用 `cond_broadcast` 是因为读者与写者都有可能等待在 `has_writer` 上, 因此两者都需要被唤醒。如果使用 `cond_signal`, 有可能只唤醒了一个读者, 那么等待在 `has_writer` 上的写者将永远不会被唤醒, 从而陷入无限等待。

从上面介绍的两种读写锁的实现中我们可以发现, 偏向读者的读写锁能大幅提高读者之间的并行度, 但是写者由于需要等待某一个时刻完全没有读者时才能更新数据, 需要面临很高的写延迟。而偏向写者的读写锁虽然避免了这个问题, 让写者能够在之前的读者完成读临界区后立刻进入, 但读者之间的并行度大幅下降。因此开发者需要根据具体场景的需求选用合适的读写锁。

10.5 案例分析: Linux 中的 `futex`

为了解决循环忙等的问题, 我们在 10.2 节介绍了条件变量。但条件变量需要与一个基于循环等待的互斥锁搭配使用。如果需要在互斥锁 (比如代码片段 10.4 中实现的自旋锁) 中使用条件变量避免循环, 那么在每次上锁时需要额外进行加锁 / 放锁操作, 这会带来不必要的开销。这种性能开销在互斥锁竞争程度低时更为显著: 之前的实现只需要一次原子操作即可上锁, 而使用了条件变量后则需要两次原子操作 (额外的用于保护条件变量的互斥锁)。另一个方案是直接使用如 `SYS_yield` 的系统调用来挂起自己。但这样挂起与唤醒时机完全是由操作系统调度器控制的, 可能出现还没挂起时锁已经释放或锁释放很久后线程才被唤醒的情况。

因此, Linux 提供了 `futex` (Fast User-space muTEX) 机制。使用 `futex` 机制实现的互斥锁能在竞争程度较低时, 直接使用原子操作完成加锁。而竞争程度较高时, 应用程序能够通过系统调用挂起并等待后续锁持有者被唤醒。`futex` 机制不仅可以避免互斥锁中的